

测试报告

1 黑盒测试报告

1.1 测试概述

本次黑盒测试面向 RAG-Knowledge-Assistant 项目展开，围绕用户管理、课程管理、课程资料管理、练习题管理、课程社区管理以及核心业务流程，对系统主要业务功能进行验证。

测试用例设计主要采用等价类划分、边界值分析、决策表法和场景法。测试执行采用“自动化接口测试为主、少量人工页面测试为辅”的方式，其中自动化测试承担主体覆盖任务，人工测试用于补充前端表单交互、PDF 预览、下载行为和页面级端到端流程验证，并结合实际运行结果与人工验证记录进行综合判断。

1.2 测试环境与工具

1.2.1 测试环境

1. 操作系统：Windows 11 开发环境。
2. 前端：React 19。
3. 后端：FastAPI。
4. 浏览器：Chrome 与 Edge。
5. 前端访问地址：`http://localhost:3000`。
6. 后端接口地址：`http://127.0.0.1:8000`。
7. 自动化测试环境：SQLite 测试库用于承载关系型业务数据；资料模块使用文档存储替身模拟文件行为。

1.2.2 测试工具

1. Codex：用于辅助生成测试计划、测试用例、自动化测试代码和测试报告整理内容。
2. pytest + FastAPI TestClient：用于执行自动化黑盒接口测试。
3. 浏览器人工操作：用于执行少量页面黑盒验证。

1.3 测试执行结果

1.3.1 自动化测试结果

自动化黑盒测试共执行 36 项，全部通过。

执行命令：

```
D:\miniconda3\envs\rag\python.exe -m pytest -q
```

执行结果：

36 passed, 9 warnings

自动化测试已覆盖用户管理、课程管理、资料管理、练习题管理、课程社区管理以及核心流程的主要黑盒接口验证，具体包括注册、登录、个人信息查看与修改、课程创建、加入与退出、资料上传、查看与删除、练习题查看与作答、社区发帖、评论、回复等主要功能点。

1.3.2 人工测试结果

人工黑盒测试共执行 10 项，全部通过。

人工测试主要补充验证了以下内容：

1. 注册页确认密码校验与角色选择校验
2. 登录后首页跳转、身份状态展示与刷新后的登录状态保持
3. PDF 资料预览效果与页面展示情况
4. 资料下载时的浏览器下载行为
5. 练习题页面反馈展示与“重新作答”后的状态恢复
6. 匿名发帖与匿名评论的展示效果
7. 回复评论后的页面定位与回显
8. 页面级核心流程端到端验证

1.3.3 综合统计

1. 自动化测试：36 项，通过 36 项。
2. 人工测试：10 项，通过 10 项。
3. 黑盒测试总执行数：46 项。
4. 黑盒测试总通过数：46 项。

自动化测试与人工测试结果一致，计划范围内的主要黑盒功能均完成验证。

1.4 缺陷与问题分析

本次黑盒测试未发现阻塞主要业务流程的严重缺陷，系统主要功能能够按需求正常运行，未出现导致系统无法使用、核心流程中断或角色权限错误的问题。

测试过程中记录到的轻微问题或风险如下：

1. 回复评论定位功能在页面滚动距离较长时，定位体验可能不够稳定。该问题未导致功能失效，因此未判定为失败项，但建议在后续版本中优化页面滚动与定位交互体验。
2. 自动化测试执行中出现若干技术栈弃用警告。相关警告主要来自 Pydantic、SQLAlchemy 和 FastAPI 的版本兼容提示，不影响本次黑盒测试通过结果，但说明项目后续存在升级维护需求。

1.5 测试结论

本次黑盒测试结果表明，RAG-Knowledge-Assistant 项目在当前测试范围内整体表现良好，主要业务功能满足需求说明文档要求，教师与学生角色权限控制正确，课程平台核心业务流程能够顺利完成。

综合自动化测试与人工测试结果，可以认为该系统在黑盒测试范围内达到了课程作业要求，具备较好的功能完整性和可用性。

2 白盒测试报告

2.1 测试概述

本次白盒测试重点覆盖用户与认证、课程管理、课程资料管理、练习管理和课程社区管理五类业务。测试目标是基于代码结构验证关键逻辑的正确性与稳健性，并满足课程作业对白盒测试方法与覆盖率指标的要求。

白盒测试的执行策略：优先通过后端接口触发真实业务调用链，贯通 `controller`、`service`、`repository`、`dao` 等后端层次，仅对外部系统依赖（如文档存储客户端）使用替身隔离，以保证测试结果既具可重复性又具工程真实性。

本阶段白盒测试由成员 B 与成员 C 分工完成。成员 B 负责用户与认证、课程管理、课程资料管理相关代码；成员 C 负责练习管理与课程社区管理相关代码。两部分均采用代码覆盖分析、基本路径覆盖和数据流分析三类方法，测试组织方式和结果统计口径保持一致。

2.2 测试环境与工具

2.2.1 测试环境

1. 运行环境：Python 3.11 / Python 3.12。
2. 后端框架：FastAPI。
3. 关系型数据测试环境：SQLite 内存数据库（测试会话）。
4. 文档存储测试环境：Mongo/GridFS 外部依赖替身。

2.2.2 测试工具

1. Codex：辅助设计测试用例、分析未覆盖路径和整理报告。
2. pytest：执行自动化白盒测试。
3. FastAPI TestClient：通过接口请求驱动后端完整调用链。
4. pytest-cov：统计覆盖率并输出未命中行。

2.3 测试对象说明

白盒测试覆盖如下代码模块：

1. 用户与认证调用链: LoginController → LoginService → UserRepository + UserAuthRepository; UserController → UserService → UserRepository + UserAuthRepository。
2. 课程管理调用链: CourseController → CourseService → CourseRepository + UserRepository。
3. 资料管理调用链: rag_controller → DocumentDAO → document_repository。
4. 练习管理调用链: ExerciseController、ExerciseService 与 ExerciseRepository。
5. 课程社区调用链: PostController、PostService、PostRepository 与 CommentRepository。

2.4 测试方法说明

本阶段白盒测试主要采用代码覆盖分析、基本路径覆盖和数据流分析三种方法。结合本项目后端调用链特点，各方法表现出如下特点：

2.4.1 代码覆盖分析特点

1. 优点：可量化评估测试充分性，能够快速识别未命中模块与薄弱分支，便于进行增量补测。
2. 局限：覆盖率反映“代码是否执行”，不直接保证业务语义正确；高覆盖不等于缺陷风险为零。
3. 适用性：适合作为本阶段总体进度与质量基线，用于指导接口驱动用例迭代。本次测试覆盖注册登录、课程状态流转、资料上传下载、习题集维护、帖子评论等主要语句与分支。

2.4.2 基本路径覆盖特点

1. 优点：对登录、课程、资料、练习、社区等条件组合较多的流程有较好约束能力，能够系统性覆盖成功、失败与异常路径。
2. 局限：路径数量随条件增加而增长较快，需要结合优先级聚焦关键业务路径，否则维护成本较高。
3. 适用性：适合核心流程与权限相关逻辑，尤其适用于角色差异、资源存在性、状态流转等场景。

2.4.3 数据流分析特点

1. 优点：能够验证关键变量在跨层调用中的定义-使用一致性，擅长发现“参数传递正确但语义使用错误”的问题。
2. 局限：需要明确变量生命周期与调用边界，分析成本高于常规接口通断测试。
3. 适用性：适合用户身份与角色、课程标识、文件标识、密码哈希、习题集标识、题目选项与答案、检索关键字、父评论标识和匿名标记等关键变量链路验证。

综合而言，三种方法形成互补关系：代码覆盖用于量化监控，基本路径用于保障流程完整性，数据流分析用于校验跨层变量一致性，共同支撑本阶段白盒测试结论。

2.5 测试执行结果

2.5.1 执行命令与通过情况

成员 B 执行命令：`pytest tests/ -q`

成员 B 测试结果：16 passed。

成员 C 执行命令如下：

```
pytest tests/test_exercise_module.py tests/test_post_module.py -q
```

成员 C 共执行 26 个白盒测试用例，其中 23 个通过，3 个失败。3 个失败用例均为缺陷触发型用例，失败原因是当前系统缺少对应业务校验，并非测试环境或测试代码异常。

2.5.2 覆盖率结果（本次执行）

成员 B 覆盖率统计命令如下：

```
pytest tests/ -q --cov=controller --cov=service \
--cov=repository --cov=dao --cov-report=term-missing
```

成员 B 覆盖结果如下：

代码文件名	覆盖率	未覆盖行数
controller/CourseController.py	91%	18-22, 99, 114, 143
controller/LoginController.py	85%	12-16
controller/UserController.py	100%	—
controller/__init__.py	100%	—
controller/rag_controller.py	100%	—
dao/__init__.py	100%	—
dao/document_dao.py	100%	—
repository/CourseRepository.py	98%	34
repository/UserAuthRepository.py	100%	—
repository/UserRepository.py	92%	19, 29
repository/__init__.py	100%	—
repository/document_repository.py	97%	38
service/CourseService.py	100%	—
service/LoginService.py	100%	—
service/UserService.py	100%	—
service/__init__.py	100%	—
成员 B 负责代码小计	达到目标	少量非核心语句未覆盖

成员 C 覆盖率统计命令如下：

```
pytest tests/test_exercise_module.py tests/test_post_module.py -q \
--cov=controller/ExerciseController.py --cov=controller/PostController.py \
--cov=service/ExerciseService.py --cov=service/PostService.py \
--cov=repository/ExerciseRepository.py --cov=repository/PostRepository.py \
--cov=repository/CommentRepository.py --cov-report=term-missing
```

成员 C 针对练习与社区模块的覆盖率统计结果如下：

代码文件名	覆盖率	未覆盖行数
controller/ExerciseController.py	100%	—
controller/PostController.py	100%	—
repository/CommentRepository.py	100%	—
repository/ExerciseRepository.py	100%	—
repository/PostRepository.py	100%	—
service/ExerciseService.py	100%	—
service/PostService.py	100%	—
成员 C 负责代码小计	100%	—

其中成员 C 负责模块总计 221 条语句、22 个分支，语句覆盖率和分支覆盖率均达到 100%。该结果说明练习管理与课程社区管理中的成功路径、异常路径、权限分支、匿名/实名映射、分页过滤、评论回复等关键逻辑均已被执行。

2.6 未覆盖代码分析

成员 B 负责模块中仍存在少量未覆盖语句，主要并非测试遗漏或核心业务逻辑缺失，具体可分为以下三类：

- **防御性分支不可触发**：当前调用链中其它层会提前进行校验，提前返回或抛出异常，导致下游用于兜底处理的部分分支不可达。例如 service 层直接抛出 `HTTPException`，该异常由 FastAPI 框架统一捕获并转换为响应，controller 层中针对返回值为 `None` 的防御分支因此无法执行。
- **数据库生成器语句未覆盖**：LoginController 与 CourseController 中 `get_db()` 生成器的 `SessionLocal()` 与 `db.close()` 未被覆盖，原因是测试为保证可重复性采用 SQLite 测试会话并通过依赖覆盖替换了默认数据库依赖，从而未执行生产环境函数体。
- **文档存储仓储层的生命周期函数未覆盖**：仅在应用启动/关闭事件中调用，不属于常规接口请求路径。

成员 C 负责的练习与社区模块本次统计范围内未覆盖行数为 0。需要说明的是，覆盖率结果仅说明代码路径已被执行，不代表业务规则完全无缺陷；本次白盒测试正是在高覆盖执行过程中发现了 3 个输入校验与数据一致性问题。

2.7 缺陷与问题分析

本次白盒测试共发现 3 个明确缺陷，均来自练习与社区模块的缺陷触发型用例。

2.7.1 系统允许上传空题目集

1. 对应测试用例：`test_upload_empty_question_list_should_be_rejected`。
2. 实际结果：接口返回 200 OK。
3. 预期结果：应返回 400 或 422，拒绝空题目集。
4. 原因分析：当前练习上传接口和习题集创建服务仅对请求进行接收和保存，没有对 `questions` 是否为空进行校验，导致系统可以创建没有题目的习题集。

2.7.2 系统允许对不存在的帖子发表评论

1. 对应测试用例：`test_add_comment_should_reject_missing_post`。
2. 实际结果：接口返回 200 OK。
3. 预期结果：应返回 400 或 404，拒绝对不存在的帖子发表评论。
4. 原因分析：当前 `PostController.add_comment()` 在调用评论创建逻辑前未验证 `post_id` 是否存在，可能导致非法评论数据被接受，影响社区数据一致性。

2.7.3 系统允许回复不存在的父评论

1. 对应测试用例：`test_add_comment_should_reject_missing_parent`。
2. 实际结果：接口返回 200 OK。
3. 预期结果：应返回 400 或 404，拒绝回复不存在的父评论。
4. 原因分析：当前 `PostController.add_comment()` 未验证 `parent_id` 是否对应真实评论记录，仓储层直接保存该字段，可能导致评论树出现断裂。

2.8 测试结果与分析

从白盒测试结果来看，用户与认证、课程管理、课程资料管理、练习管理和课程社区管理的主要业务路径均已得到有效验证。成员 B 负责模块的测试用例全部通过，核心业务逻辑功能正常；成员 C 负责模块的主要功能路径可正常运行，同时通过缺陷触发型用例发现了 3 个实际业务校验缺陷。

整体来看，本次白盒测试采用统一的方法与工具完成了代码覆盖、基本路径和数据流验证，主要业务分支与常见异常路径均经过验证，测试质量达到课程作业要求。对于仍未命中的少量语句，已通过代码与调用链分析确认其主要来源；对于已发现的缺陷，后续应优先补充空题目集校验、评论目标存在性校验和回复父评论存在性校验。